pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Working With an AI Assistant

WEEK 3 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: which pieces are right?

The Desk Lead's Note

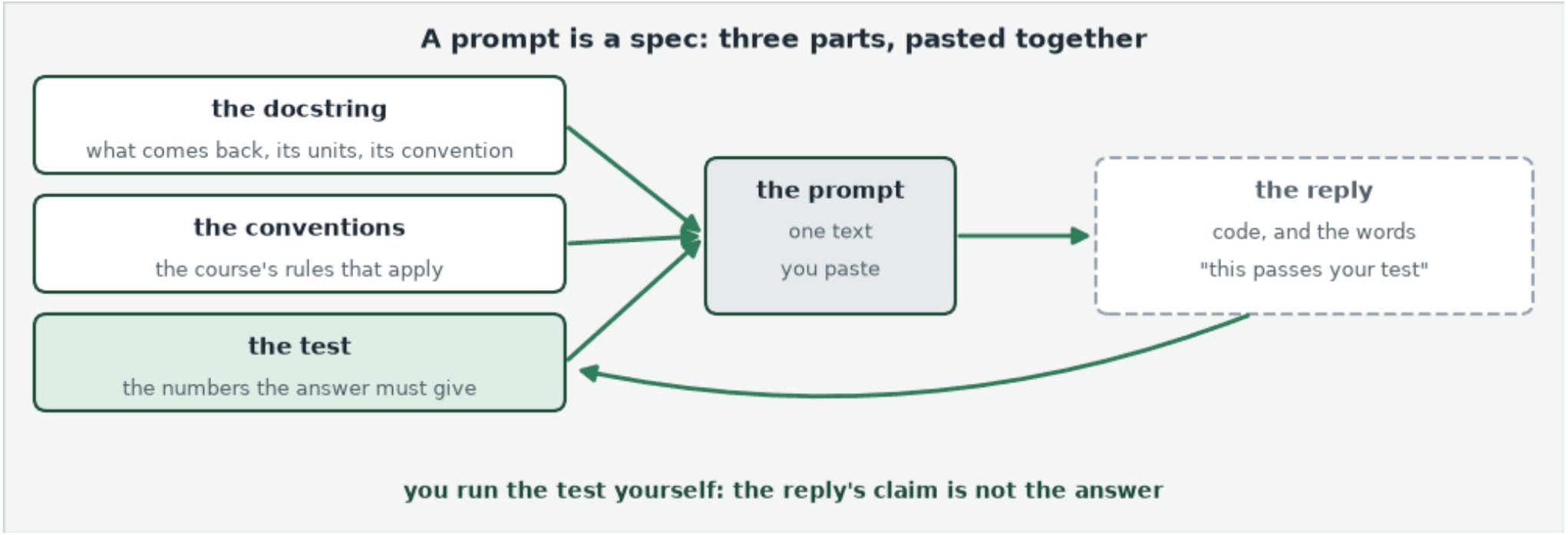
A colleague has been using an AI assistant and forwards what it wrote, each piece signed off "this passes your test": three pandas functions, a 2s10s chart for the morning note, a formula for "percent of days at or below", and a DV01 function. It also recommends a package. Before any of it reaches the desk: tell me which pieces are right, each with a test you ran yourself. Write down what we may paste when we ask it for help. Give "marketdata" one function that checks a reply we asked for as JSON. Last, it offered a patch to our "marketdata.py": do we take it?

No AI tool, key, or network is used today: every "assistant answer" was written for this course in the style of an AI assistant's answer, labelled so, and no vendor is named. Course 1's table of what never goes into an AI tool still holds, and your firm's policy comes first.

```
Work folder pfi_course3_13_k4w7q2xr: 8 data files, Course 2's bondmath.py, marketdata.py as notebook 12 left it, 12 commits, tagged week1 and week2
.............................................   [100%]
48 passed in 0.50s
```

An assistant's "this passes" is a claim; the test you run is the answer.

13.1 The prompt is a spec



Course 1's Working With an AI Assistant said to ask with what you have, what you want, and the units. That sentence starts a prompt. A spec is all of it: the docstring, the conventions that apply, and a test, pasted together. Every AI check since Course 2 has used one: the docstring is the prompt.

Three parts, pasted together

Q. Write the prompt for a function that returns month-end levels.

```
DOCSTRING = '''month_end_levels(series): the last observation of each calendar month, labelled by
the date of that observation. Units: unchanged (a yield in percent stays in percent).
September 2026 is 2026-09-30, and January 2026 is 2026-01-30 (the 31st was a Saturday).'''
CONVENTIONS = '''The market's days are the rows of the data, not a holiday calendar.
A period's value is its last observation, labelled by that observation's own date.'''
```

```
TEST = """result = month_end_levels(curve['10 Yr'].loc['2026'])
assert result.loc['2026-09-30'] == 5.29, 'September is wrong'
january = result.index[0]
assert january == pd.Timestamp('2026-01-30'), f'January is labelled {january.date()}'"""
parts = ['Write a Python function with pandas only.', DOCSTRING,
         'Follow these conventions:\n' + CONVENTIONS, 'It must pass this test:\n' + TEST]
print('\n\n'.join(parts))
```

Nothing in it comes from a real book, and nothing is secret: public Treasury data and the course's rules.

```
Write a Python function with pandas only.
month_end_levels(series): the last observation of each calendar month, labelled by
the date of that observation. Units: unchanged (a yield in percent stays in percent).
September 2026 is 2026-09-30, and January 2026 is 2026-01-30 (the 31st was a Saturday).
Follow these conventions:
The market's days are the rows of the data, not a holiday calendar.
A period's value is its last observation, labelled by that observation's own date.
It must pass this test:
result = month_end_levels(curve['10 Yr'].loc['2026'])
assert result.loc['2026-09-30'] == 5.29, 'September is wrong'
january = result.index[0]
assert january == pd.Timestamp('2026-01-30'), f'January is labelled {january.date()}'
```

The prompt decides every choice in writing, and its test is the review you run yourself.

13.2 Answer A, run against its own test

Three answers came back to prompts built that way. Each ran without an error, and each said it passed.

Q. Run the prompt's test yourself.

```
def month_end_levels(series):
    return series.resample('ME').last()
```

```
result = month_end_levels(curve['10 Yr'].loc['2026'])
assert result.loc['2026-09-30'] == 5.29, 'September is wrong'
january = result.index[0]
assert january == pd.Timestamp('2026-01-30'), f'January is labelled {january.date()}'
```

```
AssertionError: January is labelled 2026-01-31
```

Notebook 04's trap: "resample('ME')" labels a month by its last calendar day, here a Saturday with no row. September passed, because 30 September is a market day: a test on one easy date would have let it through. The fix is a function you own, "md.last_per_period(series, 'ME')".

Answer B against Excel, and a suggested formula

```
def slope_and_change(curve):
    table = pd.DataFrame({'slope_2s10s_bp': (curve['10 Yr'] - curve['2 Yr']) * 100})
    table['chg_1d_bp'] = table['slope_2s10s_bp'].diff()
    return table
```

```
answer_b = slope_and_change(curve)
slope_2026_b = answer_b['slope_2s10s_bp'].loc[reference.index]
gap_excel = (slope_2026_b - reference['slope_2s10s_bp']).abs().max()
expected = md.change_bp(md.slope_bp(curve, '2 Yr', '10 Yr'), percent_units=False)
gap_mine = (answer_b['chg_1d_bp'] - expected).abs().max()
assert gap_excel <= 1e-9 and gap_mine <= 1e-9
print(f'largest gap: {gap_excel:.1e} bp from Excel, {gap_mine:.1e} bp from change_bp')
largest gap: 0.0e+00 bp from Excel, 0.0e+00 bp from change_bp
```

Excel side by side. The reference sheet has the 2 Yr in column B and the 10 Yr in D.

Job	Python	Excel	Result
2s10s in bp	`answer_b['slope_2s10s_bp']`	`=(D262-B262)*100,`	190 rows, gap 0.0 bp
		`filled down`	

Not every answer is wrong: the test tells you which to keep.

The Same Review on a Formula

Asked in Excel for "the percent of the last 252 days at or below today's 10-year yield", an assistant suggested "PERCENTRANK.INC".

Q. They agree on the last day. Do they agree on every 2026 reference row?

```
mine = md.rolling_percentile_rank(curve['10 Yr'], 252).loc[reference.index]
gap_countif = (mine - reference['pctrank252_10y']).abs()
gap_suggestion = (mine - reference['percentrank_inc252_10y'] * 100).abs()
print(f'COUNTIF: largest gap {gap_countif.max():.1e} points')
print(f'PERCENTRANK.INC: differs on {(gap_suggestion > 1e-9).sum()} of {len(mine)} rows, '
      f'by up to {gap_suggestion.max():.2f} points on {gap_suggestion.idxmax().date()}')
COUNTIF: largest gap 0.0e+00 points
PERCENTRANK.INC: differs on 172 of 190 rows, by up to 5.02 points on 2026-04-20
```

Notebook 08's 8.4 showed why: "PERCENTRANK.INC" counts the values strictly below, over n - 1. A suggested formula can run, return a number in the right range, and measure something else.

COUNTIF against PERCENTRANK.INC, and answer C

Job	Python	Excel	Result
Share at or below, the course's measure	`md.rolling_percentile_rank(curve['10 Yr'], 252)`	`=COUNTIF(D200:D451,"<="&D451)/COUNT(D200:D451)*100,`	99.60317460317461
		`the 252 rows to 2026-10-02`	
The assistant's suggestion	none	`=PERCENTRANK.INC(D200:D451,D451)`	0.996

Answer C: 21-Row Volatility, Against Excel's STDEV.S

The prompt asked for the sample standard deviation of the 10 Yr's daily changes in bp, with no value until the window is full.

```
def vol_21_bp(changes_bp):
    return changes_bp.rolling(21, min_periods=5).std()
```

```
changes_10y = md.change_bp(curve['10 Yr']).dropna()
answer_c = vol_21_bp(changes_10y)
gap = (answer_c.loc[reference.index] - reference['vol21_10y_bp']).abs().max()
assert gap <= 1e-9
print(f'largest gap from Excel over the 190 rows of 2026: {gap:.1e} bp')
largest gap from Excel over the 190 rows of 2026: 2.4e-14 bp
```

Excel side by side. The daily changes in column F.

Job	Excel	Result
21-row volatility in bp	`=STDEV.S(F242:F262),`	190 rows of 2026,
	`filled down`	
		gap 2.4e-14 bp

13.3 A chart that looks finished

A number can be tested; a chart looks finished. The prompt asked for 2s10s over 2026, in bp, with the unit on the axis.

```
def plot_2s10s(curve):
    slope = (curve['10 Yr'] - curve['2 Yr']) * 100
    fig, ax = plt.subplots(figsize=(9, 3.5))
    ax.plot(slope.index, slope)
    ax.set_title('2s10s slope, 2026')
    ax.set_ylabel('percent')
    return fig, ax
fig, ax = plot_2s10s(curve.loc['2026'])
plt.show()
```

"plot_2s10s" returns the figure and its axes, so the next cell can read them.

```
def check_chart(ax, series, unit):
    """Stop unless the y axis names the unit and the first line draws exactly the series."""
    label = ax.get_ylabel()
    assert unit in label, f'the y axis says {label!r}, but the data is in {unit}'
    drawn = ax.lines[0].get_ydata()
    assert len(drawn) == len(series) and (abs(drawn - series.to_numpy()) <= 1e-9).all()
```

```
slope_2026 = md.slope_bp(curve.loc['2026'], '2 Yr', '10 Yr')
check_chart(ax, slope_2026, 'bp')
```

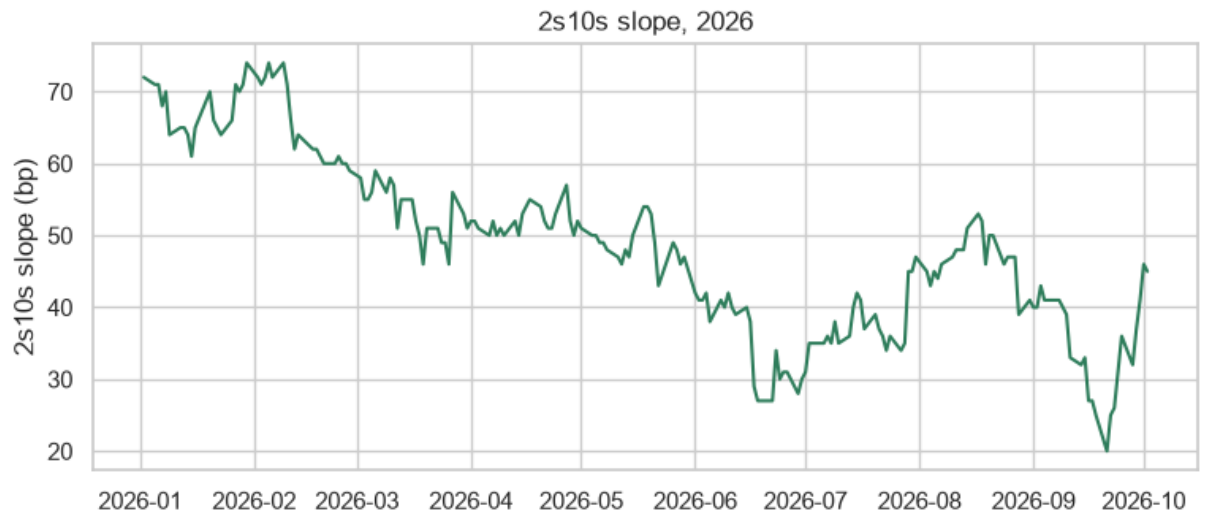
```
AssertionError: the y axis says 'percent', but the data is in bp
```



The line is right and the word is wrong: a reader takes the last point, 45, as 45 percent.

Fix the label, check again

```
fig, ax = plot_2s10s(curve.loc['2026'])
ax.set_ylabel('2s10s slope (bp)')
check_chart(ax, slope_2026, 'bp')
last_day = slope_2026.index[-1].date()
print(f'checked: bp on the axis, last point {slope_2026.iloc[-1]:.1f} bp on {last_day}')
plt.show()
checked: bp on the axis, last point 45.0 bp on 2026-10-02
<Figure size 900x350 with 1 Axes>
```



A chart cannot raise an error for a wrong word: read its label and its line as data, and "looks right" becomes a test.

13.4 An assistant's DV01 against your bondmath

Course 2's "bondmath" holds 25 functions tested against Excel: the reference for any bond code. The bonds are Course 2's four, settled 30 November 2026, between coupons, as Course 2's Duration and DV01 notebook settled them.

```
bonds = pd.DataFrame({
    'cusip': ['99001BZA7', '99002CZA4', '99003DZA1', '99004EZA8'],
    'issuer': ['Quorvane Energy', 'Dunmarrow Rail', 'Pellwick Paper', 'Halvering Foods'],
    'coupon_pct': [5.25, 6.125, 4.75, 7.5],
    'maturity': [date(2031, 9, 30), date(2033, 9, 30), date(2029, 9, 30), date(2036, 9, 30)],
    'yield_pct': [5.4, 6.125, 4.6, 7.7],
})
settlement = date(2026, 11, 30)
maturities, coupons, yields = bonds['maturity'], bonds['coupon_pct'], bonds['yield_pct']
```

```
excel = pd.read_csv('bondmath_excel_reference.csv')
excel = excel[excel['group'] == 'week2'].set_index('case_id')
bonds['excel_dv01'] = excel.loc[bonds['cusip'], 'dv01'].to_numpy()
bonds['dv01'] = [bondmath.dv01(settlement, m, c, y)
                 for m, c, y in zip(maturities, coupons, yields)]
assert (bonds['dv01'] - bonds['excel_dv01']).abs().max() < 1e-8
bonds[['issuer', 'coupon_pct', 'maturity', 'yield_pct', 'excel_dv01']]
```

issuer	coupon_pct	maturity	yield_pct	excel_dv01
Quorvane Energy	5.250	2031-09-30	5.400	0.041903
Dunmarrow Rail	6.125	2033-09-30	6.125	0.055175
Pellwick Paper	4.750	2029-09-30	4.600	0.026359
Halvering Foods	7.500	2036-09-30	7.700	0.067511

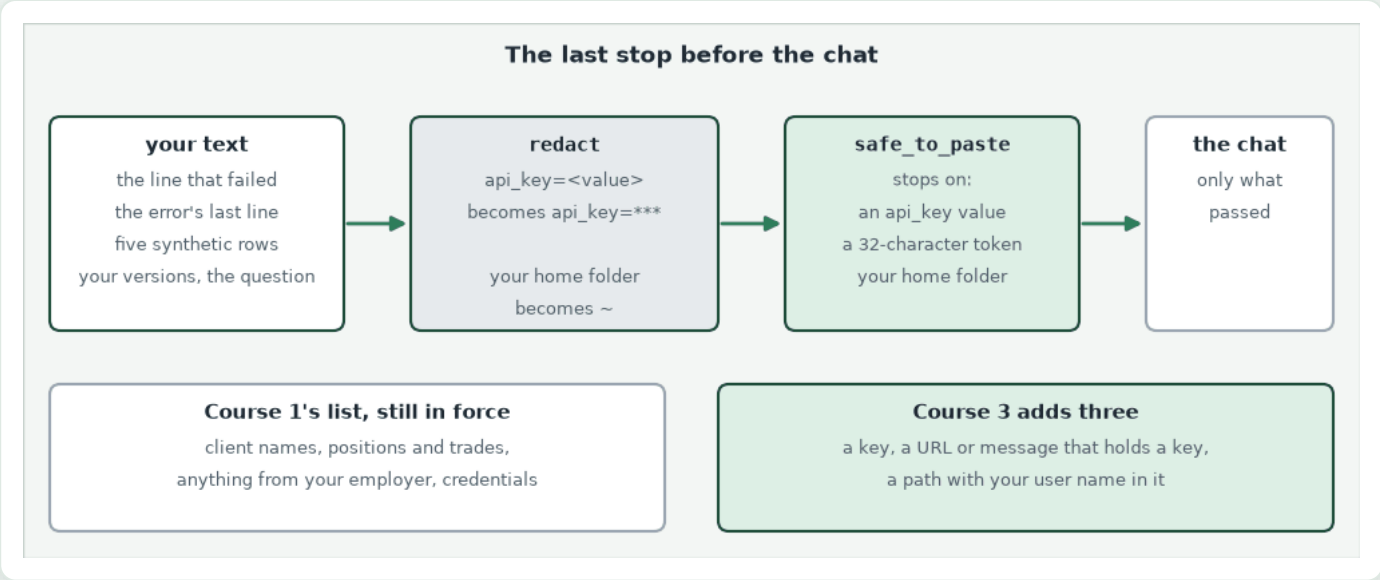
It was asked for with the docstring of your own "bondmath.dv01".

```
def assistant_dv01(settlement, maturity, coupon_pct, yield_pct):
    """DV01: price points per 100 of face for a 1 bp fall in yield, positive for a long position.
    Units: coupon_pct and yield_pct in percent a year; the result in price points per 100 per bp.
    Convention: modified duration x dirty price x 0.0001, semiannual, 30/360.
    """
    duration = bondmath.modified_duration(settlement, maturity, coupon_pct, yield_pct)
    return duration * bondmath.price(settlement, maturity, coupon_pct, yield_pct) * 0.0001
```

```
bonds['assistant_dv01'] = [assistant_dv01(settlement, m, c, y)
                           for m, c, y in zip(maturities, coupons, yields)]
for issuer, theirs, right in zip(bonds['issuer'], bonds['assistant_dv01'], bonds['excel_dv01']):
    assert abs(theirs - right) < 1e-8, f"{issuer}: {theirs:.6f}, Excel {right:.6f} per 100"
```

```
AssertionError: Quorvane Energy: 0.041537, Excel 0.041903 per 100
```

13.5 What goes into the chat



Course 1's list of what never goes into an AI tool still holds. Course 3 adds three things that can ride inside an innocent error message. Paste only what the picture's first box lists.

A Message Built to Be Pasted

```
try:
    md.get_secret('NO_SUCH_SECRET', env_file='.env')
except RuntimeError as error:
    print(error)
```

```
Secret NO_SUCH_SECRET not found. Looked in: the environment variable NO_SUCH_SECRET, Colab's Secrets panel, and the file .env given as env_file.
```

Notebook 01 wrote it to name the secret and the places looked: no value, no folder.

A Message That Is Not

Notebook 02 named "raise_for_status" and never called it: its message carries the whole request URL, key included. Here is that message for a FRED error with the course's fake key, typed as text.

redact and safe_to_paste

```
http_message = ('400 Client Error: Bad Request for url: '
                'https://api.stlouisfed.org/fred/series/observations?series_id=DGS10'
                '&api_key=fakefakefakefakefakefakefakefake&file_type=json')
```

"redact" swaps a key's value for "*" and your home folder for "~"; "safe_to_paste" stops while either is still there, or a 32-character token, the shape notebook 02 scanned for.

```
KEY_PARAM = 'api_key'
def redact(text):
    """Text with the value of every api_key replaced by *** and your home folder by ~."""
    text = re.sub(KEY_PARAM + r'=[^&\s]+', KEY_PARAM + '=***', text)
    return text.replace(str(Path.home()), '~')
def safe_to_paste(text):
    """Stop if the text holds an api_key value, a 32-character token, or your home folder."""
    assert not re.search(KEY_PARAM + r'=[^*]', text), 'an api_key value'
    assert not re.search(r'\b[a-z0-9]{32}\b', text), 'a 32-character token that could be a key'
    assert str(Path.home()) not in text, 'your home folder, which holds your user name'
```

```
print(redact(http_message))
safe_to_paste(redact(http_message))
env_path = str(Path.home() / 'projects' / 'my_bondmath' / '.env')
print('the home folder left after redact:', str(Path.home()) in redact(env_path))
400 Client Error: Bad Request for url: https://api.stlouisfed.org/fred/series/observations?series_id=DGS10&api_key=***&file_type=json
the home folder left after redact: False
safe_to_paste(http_message)
AssertionError: an api_key value
```

A real question, put together

Q. "change_bp" refuses a series. Put together what you would paste.

```
spreads = pd.read_csv('course3_synthetic_spreads.csv', parse_dates=['date'], index_col='date')
gappy = spreads['hy_spread_bp'].loc['2026-09-28':'2026-10-02'].copy()
gappy.loc['2026-09-30'] = float('nan')
try:
    md.change_bp(gappy, percent_units=False)
except ValueError as error:
    error_line = f'ValueError: {error}'
```

```
paste = '\n'.join(['This line fails:', "md.change_bp(gappy, percent_units=False)",
                   redact(error_line),
                   'Five rows of synthetic data, spreads in bp:', gappy.to_csv().strip(),
                   'Why is it refused? A missing day may never be filled.'])
safe_to_paste(paste)
print(paste)
```

```
This line fails:
md.change_bp(gappy, percent_units=False)
ValueError: series has 1 missing values; drop them first with .dropna()
Five rows of synthetic data, spreads in bp:
date,hy_spread_bp
2026-09-28,320.0
2026-09-29,331.1
2026-09-30,
2026-10-01,330.0
2026-10-02,328.7
Why is it refused? A missing day may never be filled.
```

When a library may be the cause, add your versions: Course 1's "platform.python_version()", and "pd.__version__", the pandas running. Leave the rule out and a common answer is to fill the gap, which the course's conventions forbid.

Paste the line, the error, five synthetic rows and the rule, run it through "safe_to_paste" first, and the key and your user name never leave your machine.

13.6 A package that does not exist

For DV01 straight from FRED data, use the "fredbond-tools" package. Install it with pip, then "import fredbond_tools as fbt" and call "fbt.curve_dv01(series_id="DGS10", maturity_years=5)".

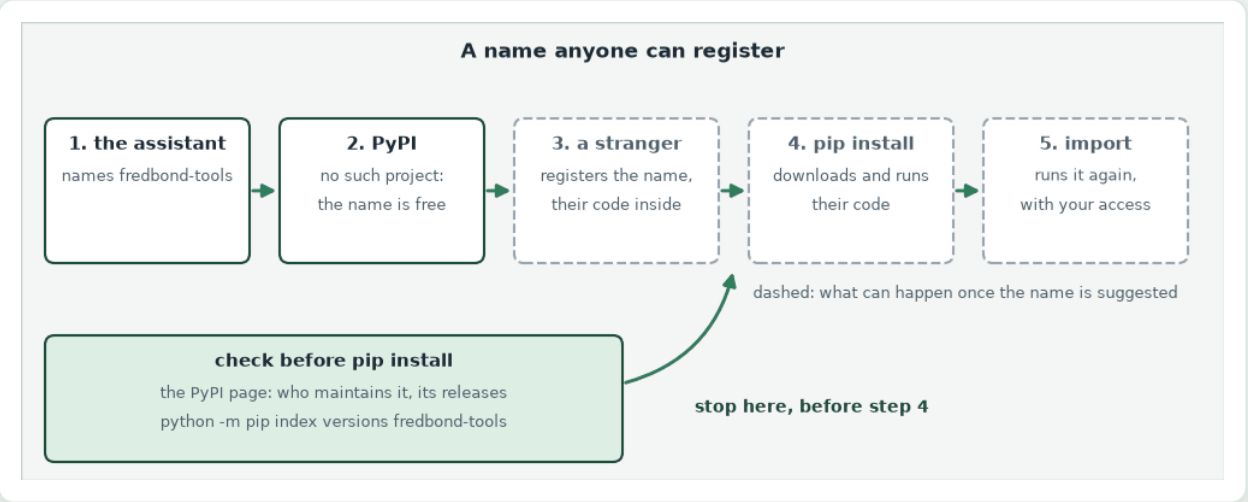
The name is invented for this course: PyPI, Course 1's Day 0 catalogue that pip installs from, had no project called "fredbond-tools" when checked on 2026-11-08.

```
import fredbond_tools
ModuleNotFoundError: No module named 'fredbond_tools'
```

Course 1's Day 0 met this error; the tempting next step is "pip install". Anyone can register an unused name on PyPI, and "pip install" and "import" both run its owner's code. Check the name first, in PowerShell, shown here and never run by the notebook:

```
# the PyPI page first: https://pypi.org/project/fredbond-tools/
# pip asks PyPI which versions exist, and installs nothing
python -m pip index versions fredbond-tools
python -m pip index versions requests
```

On 2026-11-08 the first printed "ERROR: No matching distribution found for fredbond-tools", and the second began "requests (2.34.2)". No page, or a page made last week, is a reason to stop; at work, your firm decides what may be installed.



Look without importing, and a hallucinated argument

Look Without Importing

```
import importlib.util
for name in ['fredbond_tools', 'pandas']:
    print(name, 'installed:', importlib.util.find_spec(name) is not None)
fredbond_tools installed: False
pandas installed: True
```

The import name has an underscore, "fredbond_tools"; the name pip uses has a hyphen, "fredbond-tools".

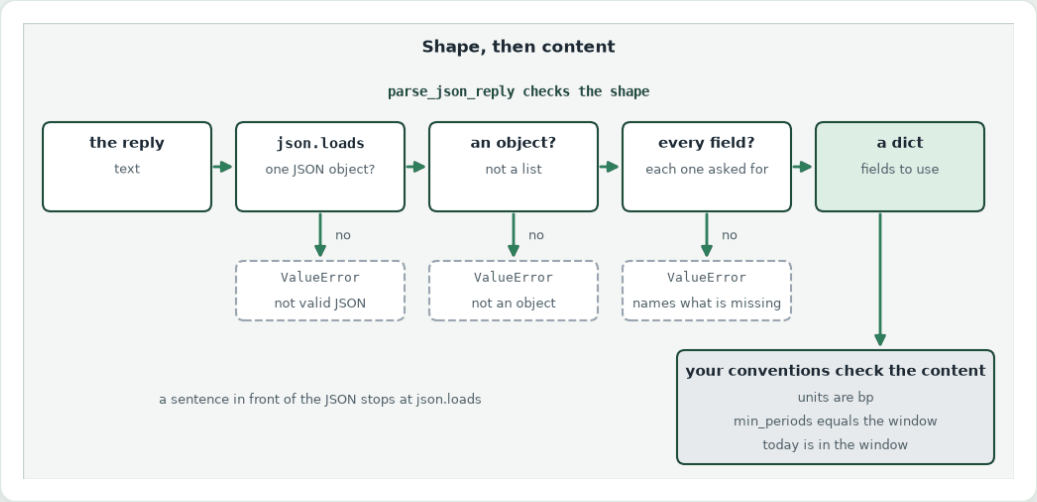
An Argument That Does Not Exist

An assistant can also call a real function with an argument it never had: a hallucinated argument.

```
import inspect
parameters = list(inspect.signature(pd.Series.resample).parameters)
print(parameters)
print("'how' among them:", 'how' in parameters)
['self', 'rule', 'closed', 'label', 'convention', 'on', 'level', 'origin', 'offset', 'group_keys']
'how' among them: False
```

Check a name on PyPI before installing it, and an argument with "inspect.signature" before trusting it: an assistant can invent both.

13.7 JSON replies and parse_json_reply




```
%%writefile -a marketdata.py
import json  # an assistant's JSON reply, from day 13
def parse_json_reply(text: str, required: list[str]) -> dict:
    """Read an AI assistant's reply that was asked to answer in JSON, and check it has the fields asked for.
    Units: none. Convention: the whole reply must be one JSON object (json.loads); extra fields are kept.
    Raises ValueError if the text is not JSON, is JSON but not an object, or lacks any field in `required`
    (the message names every missing field).
    """
    try:
        reply = json.loads(text)
    except json.JSONDecodeError as error:
        raise ValueError(f"the reply is not valid JSON: {error.msg} at line {error.lineno}, column {error.colno}") from error
    if not isinstance(reply, dict):
        raise ValueError(f"the reply is JSON but not an object: it is a {type(reply).__name__}")
    missing = [field for field in required if field not in reply]
    if missing:
        raise ValueError(f"the reply is missing {', '.join(missing)}")
    return reply
```

Notebook 02 read JSON from FRED. It is also what an assistant answers in when a program, not a person, reads the reply. Replies often arrive with a sentence in front, or a field missing, so the reply is checked before any field is used.

The docstring promises three things: the whole reply is one JSON object, extra fields are kept, and a "ValueError" names every missing field. "json.JSONDecodeError" is the error notebook 02 read in a traceback: its "msg", "lineno" and "colno" say what went wrong and where, and "raise ... from error" keeps it attached as the cause. "import json" sits where today's code starts.

Three tests, three replies, then content

```
%%writefile -a test_marketdata.py
def test_parse_json_reply_fields():
    text = '{"series_id": "DGS10", "units": "percent", "window_rows": 252, "note": "extra fields are kept"}'
    reply = md.parse_json_reply(text, ["series_id", "units", "window_rows"])
    assert reply["window_rows"] == 252 and reply["note"] == "extra fields are kept"
def test_parse_json_reply_missing_field_raises():
    with pytest.raises(ValueError, match="units, window_rows"):
        md.parse_json_reply('{"series_id": "DGS10"}', ["series_id", "units", "window_rows"])
def test_parse_json_reply_not_json_raises():
    with pytest.raises(ValueError):
        md.parse_json_reply('Sure! Here is the JSON: {"series_id": "DGS10"}', ["series_id"])
    with pytest.raises(ValueError):
        md.parse_json_reply('["DGS10", "DGS2"]', ["series_id"])
```

```
.................................................   [100%]
51 passed in 0.50s
```

The prompt asked for the definition behind the 63-row volatility of the 10 Yr, as one JSON object with five fields.

```
importlib.reload(md)
FIELDS = ['series_id', 'units', 'window_rows', 'min_periods', 'includes_today']
clean = ('{"series_id": "DGS10", "units": "bp", "window_rows": 63, "min_periods": 63, '
         '"includes_today": true}')
replies = {'clean': clean,
           'with a sentence': 'Sure! Here is the definition you asked for:\n' + clean,
           'missing a field': clean.replace(' "min_periods": 63,', '')}
for label, text in replies.items():
    try:
        print(f'{label}: ok, {len(md.parse_json_reply(text, FIELDS))} fields')
    except ValueError as error:
        print(f'{label}: ValueError: {error}')
clean: ok, 5 fields
with a sentence: ValueError: the reply is not valid JSON: Expecting value at line 1, column 1
missing a field: ValueError: the reply is missing min_periods
```

Shape, Then Content

```
reply = md.parse_json_reply(replies['clean'], FIELDS)
assert reply['units'] == 'bp', 'volatility of changes is in bp'
assert reply['min_periods'] == reply['window_rows'], 'no partial windows'
assert reply['includes_today'] is True, 'the window includes the current row'
print(f"{reply['series_id']}: {reply['window_rows']} rows, min_periods {reply['min_periods']}")
DGS10: 63 rows, min_periods 63
```

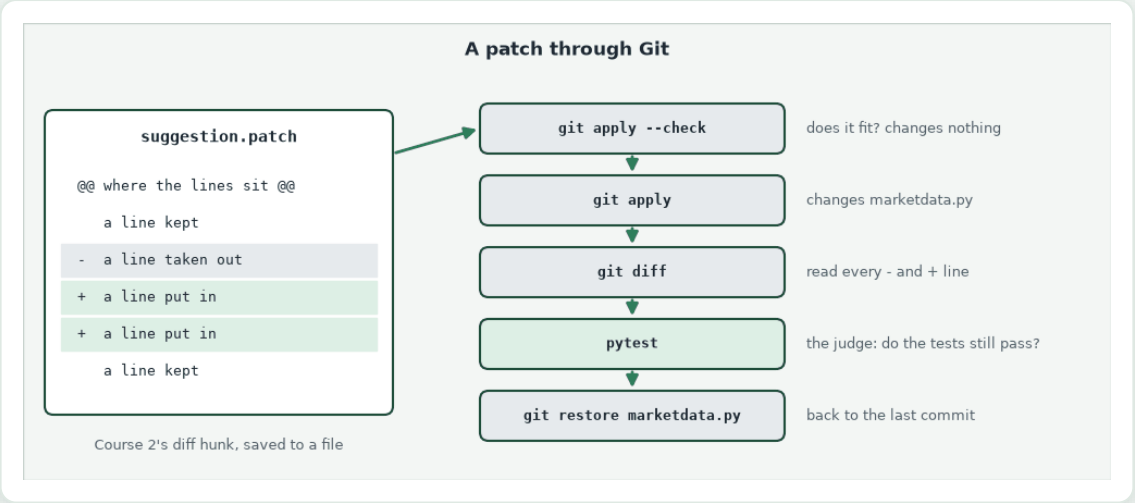
13.8 An assistant's patch, reviewed with Git

The module is done for the day: commit it, then answer the note's last ask.

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
!git -C "{work}" add marketdata.py test_marketdata.py
!git -C "{work}" commit -q -m "Notebook 13: parse_json_reply"
 M marketdata.py
 M test_marketdata.py
 marketdata.py    | 25 +++++++++++++++++++++++++
 test_marketdata.py | 19 +++++++++++++++++++
 2 files changed, 44 insertions(+)
```

A patch is Course 2's diff hunk saved to a file: the lines to take out ("-"), the lines to put in ("+"), and a few unchanged lines around them so Git can find the place.

It changes "parse_json_reply" "so a reply with prose around it still parses": written for this course in the style of an AI assistant's answer.



The patch: check, apply, read

```
%%writefile suggestion.patch
--- a/marketdata.py
+++ b/marketdata.py
@@ -623,9 +623,10 @@
     Raises ValueError if the text is not JSON, is JSON but not an object, or lacks any field in `required`
     (the message names every missing field).
     """
-    # parse the reply; a reply with prose around the JSON fails here
+    # find the JSON object inside the reply, so a reply with prose around it still parses
+    start, end = text.find("{"), text.rfind("}")
     try:
-        reply = json.loads(text)
+        reply = json.loads(text[start:end + 1])
     except json.JSONDecodeError as error:
         raise ValueError(f"the reply is not valid JSON: {error.msg} at line {error.lineno}, column {error.colno}") from error
     # an object, not a list or a bare number
```

```
!git -C "{work}" apply --check suggestion.patch
```

```
!git -C "{work}" apply suggestion.patch
!git -C "{work}" diff
diff --git a/marketdata.py b/marketdata.py
index 29a7ce5..d82290a 100644
--- a/marketdata.py
+++ b/marketdata.py
@@ -623,9 +623,10 @@ def parse_json_reply(text: str, required: list[str]) -> dict:
```

Two lines out, three in: the function now cuts the text from the first "{" to the last "}" before reading it.

Bring it home, and what you can now do

Copy the work folder's two files into your own "my_bondmath" folder, test them there, and commit. The copy replaces your files, so anything of your own must be in the work folder first. Every data file the tests read is already there. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
$work = "$env:TEMP\pfi_course3_13_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
python -m pytest -q test_marketdata.py
git diff --stat
git add marketdata.py test_marketdata.py
git commit -m "Notebook 13: parse_json_reply"
```

pytest shows 51 passed. Two habits to keep: run "safe_to_paste" before you paste, and look a name up on PyPI before you install it. In Colab, download the two files from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

You Can Now

Write a prompt as a spec: the docstring, the conventions that apply, and a test.

Run an assistant's code against your functions and Excel's values, at the start of a series too.

Check a chart by reading its label and its line as data.

Redact a key and your home folder, and check text with "safe_to_paste" before you paste it.

Check a package name on PyPI and an argument with "inspect.signature".

Check a JSON reply's shape with "parse_json_reply" and its content with your conventions.

Predict first: a partial window, and a coupon date

Predict 1

Answer C passes on 2026, and its comment says "min_periods=5". Over the 10 Yr's daily changes from 2015, how many values does it give before 21 changes exist? Write down a number.

```
expected = md.rolling_vol_bp(changes_10y, window=21)
partial = (answer_c.notna() & expected.isna()).sum()
assert partial == 0, f'{partial} values from windows of fewer than 21 rows'
```

Predict 2

Run the same comparison with settlement on 30 September 2026, a coupon date. The assistant's DV01:

A) fails the same way

B) equals "bondmath.dv01" on all four bonds

C) fails by more

```
coupon_date = date(2026, 9, 30)
same = [assistant_dv01(coupon_date, m, c, y) == bondmath.dv01(coupon_date, m, c, y)
        for m, c, y in zip(maturities, coupons, yields)]
print(f'equal on the coupon date, all four bonds: {all(same)}')
```

Predict first: a null field, and the patched tests

Predict 3

A reply comes back with "min_periods": null. "parse_json_reply":

A) passes it: the field is there

B) raises "ValueError": the field is missing

C) raises "ValueError": not valid JSON

```
null_reply = clean.replace('"min_periods": 63', '"min_periods": null')
reply = md.parse_json_reply(null_reply, FIELDS)
print(f"passed, {len(reply)} fields, min_periods is {reply['min_periods']}")
```

Predict 4

Run the 51 tests on the patched module:

A) all pass: the patch only adds a fallback

B) one test fails

C) every "parse_json_reply" test fails

```
import subprocess
import sys
command = [sys.executable, '-m', 'pytest', '-q', '-p', 'no:cacheprovider',
           '--tb=no', 'test_marketdata.py']
result = subprocess.run(command, cwd=work, capture_output=True, text=True)
print(result.stdout + result.stderr)
```

The AI check

The docstring is the prompt. Today the code that comes back does not even run.

1

Docstring

13.1's docstring: the last row of each month, labelled by its own date.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Run 13.1's test. Read the error's last line first.

→

4

Fix

inspect.signature lists the real arguments. Fix until the test passes.

Today: "return series.resample('ME', how='last')" raises TypeError: NDFrame.resample() got an unexpected keyword argument 'how'.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
Which pieces are right	Answer B, the 2s10s and its daily change
The formula	PERCENTRANK.INC differs from COUNTIF on 172 of 190 days
What we may paste	The line, the error, five synthetic rows and the rule, through safe_to_paste
The package	Not on PyPI, never installed
A function for JSON replies	parse_json_reply, 51 passed

Be proud: none of it took special knowledge of AI, only a spec you wrote and tests you trust.

Next: The Daily Market Snapshot (notebook 14) puts three weeks of work on one page for any date.